iOS Dev Guide - TextField (with TextInputHelper)

Created by Peggy Liu, last modified on 13 Jan 2021

Using the TextField & TextInput Helper

All TextFields must have a TextInputHelper.

The TextField is the component consisting of a TextField and labels that contain the top text, help text and error messages (from associated TextInputHelper).

The TextInputHelper is essentially the model of the TextField that contains the data/text for the TextField (top text, help text and error messages)

If you need to create a new InputHelper click here

View sample implementation on the Catalogue app here: https://git02.ae.sda.corp.telstra.com/projects/ABLE/repos/able-ios-catalogue-app/browse/AbleCatalogue/Application/Application/Pattern/TextField

View sample Implementation in MyTelstra: https://git02.ae.sda.corp.telstra.com/projects/MYT/repos/mytelstra-ios-app/pull-requests/7119/overview

Video describing how to use TextField in app starting at 17:28: https://web.microsoftstream.com/video/9d05d2f3-c23e-4854-8388-87fc6c5d4db0

1. Identify the Type of TextInputHelper needed for the TextField. Pick from here: Text Input Helper - Library
   The default one to go with is InputHelperName - this one has no character or validations defined.
   Need a new TextInputHelper? Scroll down to "Creating a new TextInputHelper"

   Code:
2. Create Input Helper E.g.:

   Simple Constructor

   ```
   let emailHelper = InputHelperEmail { validationResult in

   //code here for what happens after Field passes validation of input

           }
   ```

   If it is an optional TextField, pass the isRequired argument:

   ```
   let emailHelper = InputHelperEmail(isRequired: false) { validationResult in

   //code here for what happens after Field passes validation of input

           }
   ```

   For Names or Account number there is additional parameter (nameHelperType or AccountNumberHelperType):

   ```
   let givenHelper = InputHelperName(nameHelperType: .givenname) { validationResult in

   //code here for what happens after Field passes validation of input

           }
   ```

3. Create TextField:

   Simple Constructor

   ```
   let textField = TextField()
   ```

4. Associate the Textfield with corresponding TextInputHelper:

   Setup InputHelper

   ```
   textField.setup(helper: emailInputHelper)
   ```

5. If you want to trigger Textfield validations on submit you can use this method to trigger validation or show error message

   to trigger validation or show error message

   ```
   textField.showFieldValidationMessage(for: validationResult, onEditing: false)
   ```

6. Customise Textfield:
   By using the inputHelpers, the text is already set, but if you must change here is soem sample customisations:

   Customise TextField

   ```
   emailHelper.topText = "Secret Email"
   ```

```
emailHelper.helpText = "please use an email that no one knows about"

emailHelper.validationDictionary[.submitError1] =  "This email address has already been taken" //custom error messag
```

7. If you want to **prefill the textfield,** you can set the prefilled content with this:

Prefilling Field

```
emailHelper.fieldText = "prefilled email"
```

No labels

Atlassian